

05 — Questions / Réponses techniques (soutenance DEV) — ÉTENDU

70+ questions. Réponses courtes, prêtes à dire. Tous les chemins sont réels.

A. Architecture & choix techniques

1. **Présente ton appli en 30 s.** UpcycleConnect : plateforme d'économie circulaire. Les particuliers donnent/vendent des objets, les pros récupèrent la matière via des conteneurs connectés. 4 rôles, back-office admin, paiement Stripe. Déployée en Docker + Nginx + HTTPS.
2. **Pourquoi séparer front et back ?** Découplage, déploiement indépendant (2 conteneurs), l'API peut servir web ET mobile.
3. **Pourquoi Go ?** Compilé, rapide, binaire unique → image Docker `alpine` légère, typage fort, `net/http` suffisant.
4. **Pourquoi pas de framework front ?** Périmètre gérable en HTML/CSS/JS vanilla ; pas d'étape de build ; Nginx sert directement.
5. **Pourquoi MySQL ?** Données très relationnelles (users, annonces, conteneurs, commandes) → clés étrangères et jointures.
6. **Architecture en couches ?** `route/` (URLs) → `admin/` (handlers) → `bdd/` (SQL) → `models/` (structs). 1 domaine = 1 fichier par couche.
7. **Comment est organisé le code Go ?** Packages : `route` , `admin` (handlers), `bdd` , `models` , `auth` . Point d'entrée `API/main.go` .
8. **Quels design patterns ?** Séparation des responsabilités (handler ≠ accès données), middleware pour l'auth. Pas d'ORM : SQL explicite.
9. **Comment communiquent les conteneurs ?** Réseau Docker `uc_net` ; le backend joint MySQL via le nom de service `mysql` , le front joint le backend via `backend` .

B. API Go

1. **Point d'entrée ?** `API/main.go` : enregistre tous les `route.RoutesX()` puis `http.ListenAndServe(":8081", nil)` .
2. **Routing ?** `net/http` standard (Go 1.22+) avec patterns `GET /admin/users/{id}` .
3. **Comment ajouter une route ?** `http.HandleFunc("GET /chemin", handler)` dans `API/route/<domaine>.go` (+ `OPTIONS`).
4. **Lire un param d'URL ?** `r.PathValue("id")` ; query : `r.URL.Query().Get("x")` ; body : `json.NewDecoder(r.Body).Decode(&s)` .
5. **Renvoyer du JSON ?** `json.NewEncoder(w).Encode(data)` .
6. **Gérer un upload ?** `r.ParseMultipartForm` , `r.FormFile("image")` , puis `SaveUpload` (`API/admin/upload.go`).
7. **Où est la logique d'upload ?** `API/admin/upload.go` : valide extension + MIME + taille, renomme en UUID, écrit dans `UPLOAD_DIR` .
8. **Comment sont gérées les erreurs ?** `http.Error(w, "msg", http.StatusXXX)` + `log fmt.Println` .
9. **Codes HTTP utilisés ?** 200/201 succès, 400 requête invalide, 401 non authentifié, 403 rôle refusé, 404 introuvable, 500 erreur serveur.

C. Frontend

1. **Comment le front appelle l'API ?** `fetch(\ ${API_BASE_URL}/route`, {headers:{Authorization:"Bearer "+token}})`` .
2. **D'où vient API_BASE_URL ?** `Frontend/script/config.js` : `localhost + /Frontend/` → `:8081` ; sinon `origin + "/api"` .

3. **Où est le token ?** `localStorage` (`token` , `userRole` , `userId`).
4. **Comment protégez-vous une page côté front ?** `Frontend/script/auth_guard.js` (`checkSession(role)`) redirige si pas de token / mauvais rôle. (Sécurité réelle = côté serveur.)
5. **Multilingue ?** `data-i18n` + `translate.js` charge `GET /api/translations?lang=fr` et remplace les textes.
6. **Thème par rôle ?** Variable CSS `--ac` basculée par une classe `.theme-*` ajoutée selon `userRole` (pages annonces/profil).
7. **Génération PDF côté client ?** `jsPDF` dans `Frontend/script/admin/box.js` (`genererRapportLogistique`).
8. **Pagination ?** Côté client dans `Frontend/script/admin/user.js` (10 users/page, `slice`).

D. Authentification & rôles

1. **Où se fait l'authentification ?** `API/admin/users.go` (`Login`) vérifie l'email + le mot de passe puis génère un JWT (`API/auth/jwt.go`).
2. **Type de token ?** JWT HS256, clé secrète `jwtKey` dans `API/auth/jwt.go`, contient `userID` + `role` + expiration.
3. **Comment le token est vérifié ?** `VerifyTokenMiddleware` lit le header `Authorization: Bearer`, parse et valide le JWT.
4. **Comment le rôle est vérifié ?** `VerifyRoleMiddleware(next, roles...)` compare `userRole` (du contexte) à la liste autorisée.
5. **Pourquoi côté serveur ?** Le front (JS, `localStorage`) est modifiable par l'utilisateur → non fiable.
6. **Les 4 rôles ?** `Utilisateur`, `Pro`, `Salarié`, `Administrateur`.
7. **Que contient le JWT ?** `userID`, `role`, date d'expiration — pas de données sensibles.
8. **Où mettez-vous le rôle après vérif ?** Dans le `context` de la requête (`userRole`), relu par le handler.
9. **Comment gérez-vous l'expiration ?** Le token a une durée ; expiré → 401 → l'utilisateur se reconnecte.
10. **Que se passe-t-il si on modifie le JWT côté client ?** La signature ne correspond plus → rejet (401).

E. Base de données

1. **Comment l'API se connecte ?** `API/bdd/db.go` `NewDB()` → `sql.Open("mysql", DSN)`.
2. **D'où viennent les identifiants DB ?** Variables d'env (`DB_HOST`, `DB_USER` ...) ; défauts en local.
3. **Injection SQL ?** Requêtes paramétrées (?) systématiques.
4. **Où est le schéma ?** `db/init.sql` (import auto au 1er `docker compose up`).
5. **Que se passe-t-il si la DB tombe ?** Les requêtes échouent → handlers renvoient 500 + log ; le front affiche « erreur serveur ».
6. **Comment sont liées les tables ?** Clés étrangères (`annonce.id_user` → `utilisateur.id`, `annonce.id_categorie` → `categorie.id`, etc.).
7. **Utilisez-vous un ORM ?** Non, SQL explicite avec `database/sql` — plus lisible et contrôlé.
8. **Comment stockez-vous les images ?** Pas en base : chemin public en base (`annonce.image`), fichier dans le volume `/app/uploads`.
9. **Charset ?** `utf8mb4` (accents/emoji). Import à faire avec `--default-character-set=utf8mb4`.

F. Docker & déploiement

1. **Comment lancer avec Docker ?** `docker compose up -d --build` → `uc_mysql`, `uc_backend`, `uc_frontend`.
2. **Combien de conteneurs ?** 3 (MySQL, backend Go, frontend Nginx).
3. **Différence local/prod ?** `docker-compose.yml` build local ; `docker-compose-prod.yml` utilise les images `amelbdj/upcycle-*` de Docker Hub sur la VM.
4. **Comment mettez-vous à jour la prod ?** `docker build` → `docker push :vNN` → sur la VM `docker compose pull && up -d`.
5. **Le Dockerfile backend ?** Multi-stage : build Go (`golang:alpine`) → image finale `alpine` avec juste le binaire (léger + sécurisé).
6. **Comment persistent les données ?** Volumes Docker : `mysql_data` (base), `uploads_data` (fichiers).

7. **Rôle de Nginx ?** Sert les fichiers statiques, proxy `/api` → backend, proxy `/uploads`, réécrit les URLs propres.
8. **Comment prouver que ce n'est pas localhost ?** `https://upcycleconnect.pro` (IP publique, certificat HTTPS, Nginx hôte Ubuntu).
9. **Healthcheck ?** MySQL a un healthcheck ; le backend attend `service_healthy` (`depends_on`).

G. Sécurité & validations

1. **Mots de passe ?** Hashés bcrypt (`golang.org/x/crypto/bcrypt`).
2. **Validez-vous les entrées ?** Uploads (ext/MIME/taille), champs requis côté handler, requêtes paramétrées.
3. **CORS ?** Headers `Access-Control-*` + réponses `OPTIONS` sur chaque handler.
4. **Secrets (Stripe/JWT) ?** En variables d'environnement (fallback en dur dans le code pour le dev). À sortir du code en prod.
5. **Un particulier peut-il accéder à l'admin ?** Non : `VerifyRoleMiddleware("Administrateur")` renvoie 403.
6. **Protégez-vous les fichiers uploadés ?** Servis en lecture via `/uploads` ; noms UUID (non devinables).

H. Logique métier

1. **Commission ?** 5 % du montant, `API/admin/stripe.go`, prélevée via `ApplicationFeeAmount` (Stripe Connect).
2. **Upcycling Score ?** `poids_kg × coefficient(matériau)` ajouté à `utilisateur.score` (`CalculateAndAddScore`).
3. **Validation des annonces ?** Une annonce est créée « En attente », un admin la passe « Validé »/« Rejeté » (`/admin/annonces/validate|refuse`).
4. **Paieement ?** Stripe Checkout + Connect ; webhook `POST /api/stripe/webhook` confirme la commande et génère la facture PDF.
5. **Règle de dépôt d'événement ?** Le salarié doit avoir un `stripe_account_id` (`API/admin/event.go`).
6. **Génération des factures ?** `API/admin/pdf.go` (`GenerateInvoicePDF`) avec la lib `go-pdf/fpdf`, stockée dans `/app/uploads/documents`.

I. Erreurs, perfs, tests, déploiement

1. **Que faites-vous en cas de 500 ?** Lire `docker compose logs -f backend` → identifier la requête/ligne fautive.
2. **Performances ?** Go compilé, requêtes SQL ciblées, images en fichiers (pas en base), pagination côté admin.
3. **Tests automatisés ?** Non trouvés dans le code (pas de `*_test.go`) → tests manuels curl/Postman/navigateur. Axe d'amélioration assumé.
4. **Comment générer des données de test ?** Import de `db/init.sql`, ou inscription via `/register`, ou création via l'admin.
5. **Lien DEV/INFRA ?** L'appli (DEV) est conteneurisée et déployée (INFRA) sur une VM avec Nginx reverse-proxy + HTTPS + domaine.
6. **Un point faible que tu assumes ?** Les secrets en dur (fallback) et l'absence de tests unitaires ; le score qui retombe sur le coef « autre ». Je sais où et comment les corriger.
7. **Une amélioration prévue ?** Tests unitaires Go, sortie des secrets vers `.env` uniquement, corriger le calcul du score par matériau.
8. **Comment ajoutes-tu une langue ?** `POST /admin/translations/add` (import JSON) ou INSERT dans `translations` + `languages`.